

FACULTY OF ENGINEERING
DEPARTMENT OF COMPUTER ENGINEERING

2025 - 2026 SPRING

PROGRAMMING LANGUAGE PROJECT

LCS Programming Language

STUDENT ID	NAME / SURNAME
230316055	Kürşat KILIÇARSLAN
230315078	Mehmet Ali Turan
230315065	Melisa Nur Hamamcı

Table of Contents

1. Introduction
2. Base Textbook Code Used
3. Overview of the Designed Language
4. Reserved Keywords and Tokens
5. BNF / EBNF Grammar
6. Lexical Analyzer Design
7. Syntax Analyzer Design
8. State Diagrams
9. Symbol Table / Lookup Table
10. Sample Programs
11. Test Results
12. Conclusion

1. Introduction

This project presents the design and implementation of a small programming language called LCS (LanguageCompileStructure). LCS is a Turkish-inspired programming language that uses Turkish keywords for its control structures, variable declarations, and output statements. The goal of this project is to demonstrate a practical understanding of programming language concepts including formal language definition, token design, lexical analysis, and parsing.

The implementation includes a fully functional lexical analyzer (Lexer) and a recursive-descent parser (Parser), both written in Java. The lexer tokenizes LCS source code files (.txt), and the parser performs syntax analysis and basic interpretation of the tokens according to the LCS language grammar.

LCS supports variable assignments (integers and strings), arithmetic expressions with correct operator precedence, string concatenation, conditional statements (eğer/değilse), two types of loop structures (her for-loop and sürece while-loop), and a print statement (yaz). The statement terminator in LCS is the underscore character (_), which is used instead of the traditional semicolon to give the language a unique identity.

The language supports Turkish characters (ç, ğ, ı, ö, ş, ü) in both keywords and identifiers, making it accessible for Turkish-speaking programmers. The project was built by modifying and extending the sample lexical analysis and recursive-descent parsing code from Chapter 4 of Robert W. Sebesta's "Concepts of Programming Languages" (10th Edition).

2. Base Textbook Code Used

This project is based on the textbook examples presented in Robert W. Sebesta, "Concepts of Programming Languages", 10th Edition, Chapter 4: Lexical and Syntax Analysis.

The following base code was used and modified:

Lexical Analyzer (page ~172): The textbook provides a basic lexical analyzer written in C that recognizes simple tokens. We used this as a starting point and made the following modifications:

- Converted from C to Java (object-oriented approach with Lexer class)
- Added Turkish character support (ç, ğ, ı, ö, ş, ü) in identifiers and keywords
- Extended token types to include STRING_LITERAL, UNDERSCORE (statement terminator), and multi-character operators (==, !=, <=, >=)

- Implemented a HashMap-based keyword lookup table instead of simple string comparison
- Added line and column tracking for error reporting
- Added string literal recognition with quote handling

Recursive-Descent Parser (page ~182): The textbook provides a recursive-descent parser that recognizes simple expressions and statements. We used this structure and made the following modifications:

- Converted from C to Java with a Parser class
- Extended the grammar to support assignments, print statements, for-loops, while-loops, and if/else conditionals
- Added an environment (HashMap) for variable storage and basic interpretation
- Implemented operator precedence correctly with evaluateExpression (+ -), evaluateTerm (* /), and evaluateFactor (atoms)
- Added condition evaluation for comparison operators (==, !=, <, >, <=, >=)
- Implemented error recovery and meaningful error messages in Turkish

Summary of Modifications: The textbook code provided the foundational structure for both the lexer and parser. We kept the character-by-character scanning approach for the lexer and the recursive-descent parsing technique for the parser. We modified them to support Turkish keywords, additional token types, complex grammar rules, and basic interpretation capabilities. The core concepts (finite automata for lexing, BNF-driven recursive descent for parsing) remain faithful to the textbook, but the implementation has been significantly extended.

3. Overview of the Designed Language

LCS (LanguageCompileStructure) is a small, educational programming language designed with Turkish-inspired syntax. The language is intentionally simple but supports the core programming constructs required by the project specification.

Language Features

Feature	Description
Data Types	Integer literals and string literals
Variables	Dynamically typed, assigned with = operator
Arithmetic	Addition (+), subtraction (-), multiplication (*), division (/) with correct

	operator precedence
String Operations	String concatenation using + operator
Output	yaz() function for printing values
Conditionals	eğer (if) and değilse (else) blocks
Loops	her (for) loop and sürece (while) loop
Statement Terminator	_ (underscore) for top-level statements
Block Delimiters	{ } (curly braces)
Comments	Not supported in current version
Turkish Support	Full support for Turkish characters (ç, ğ, ı, ö, ş, ü) in keywords and identifiers

Keywords

The LCS language uses the following Turkish keywords:

Keyword	Equivalent	Purpose
yaz	print	Output a value to console
her	for	For-loop structure
sürece	while	While-loop structure
eğer	if	Conditional statement
değilse	else	Else branch of conditional

Operators

Arithmetic: +, -, *, /

Assignment: =

Comparison: ==, !=, <, >, <=, >=

Separators

Parentheses: ()

Braces: { }

Semicolon: ; (used inside for-loop headers)

Statement terminator: _ (underscore)

4. Reserved Keywords and Tokens

The LCS programming language supports Turkish-specific syntax and includes special characters. The following table outlines the reserved words and symbols recognized by the lexer.

Category	Token Type	Lexeme (Value)	Description
Keywords	TOKEN_KEYWORD	yaz	Used for printing output
Keywords	TOKEN_KEYWORD	her	Used to initiate a for loop
Keywords	TOKEN_KEYWORD	sürece	Used to initiate a while loop
Keywords	TOKEN_KEYWORD	eğer	Used for conditional (if) statement
Keywords	TOKEN_KEYWORD	değilse	Used for else branch
Identifiers	TOKEN_IDENTIFIER	(variable name)	User-defined, includes letters/digits/Turkish chars
Literals	TOKEN_INTEGER_LITERAL	123	Integer value
Operators	TOKEN_OPERATOR	=, +, -, *, /	Assignment and arithmetic operators
Operators	TOKEN_OPERATOR	==, !=, <, >, <=, >=	Comparison operators

Separators	TOKEN_SEPARATOR	(,), {, }, ;	Grouping, blocks, loop structure
------------	-----------------	---------------	----------------------------------

Special Tokens

Category	Token Type	Lexeme	Description
Special	TOKEN_UNDERSCORE	_	Statement terminator
Special	TOKEN_EOF	null	End of input

How keywords and identifiers are distinguished: The Lexer uses a HashMap (KEYWORDS) that maps keyword strings to TOKEN_KEYWORD. When an identifier-like token is read, it is checked against this map. If found, it is classified as TOKEN_KEYWORD; otherwise, it is TOKEN_IDENTIFIER.

5. BNF / EBNF Grammar

The following BNF grammar defines the syntax of the LCS programming language. This grammar matches the implementation in the Parser.java class.

```

<program> ::= { <statement> } <EOF>

<statement> ::= <yaz_statement> "_"
               | <assignment_statement> "_"
               | <for_loop_statement>
               | <while_loop_statement>
               | <if_statement>

<yaz_statement> ::= "yaz" "(" <expression> ")"

<assignment_statement> ::= <identifier> "=" <expression>

<for_loop_statement> ::= "her" "("
                        <assignment_no_semicolon> ";"
                        <condition_expression> ";"
                        <assignment_no_semicolon> ")" "{"
                        { <loop_body_statement> } "}"

<while_loop_statement> ::= "sürece" "(" <condition_expression> ")" "{"
                        { <loop_body_statement> } "}"

<if_statement> ::= "eğer" "(" <condition_expression> ")" "{"
                { <block_statement> } "}"
                [ "değilse" "{" { <block_statement> } "}" ]

```

```

<loop_body_statement> ::= <yaz_statement_inside_loop>
                        | <assignment_inside_loop>
                        | <if_statement>

<block_statement> ::= <yaz_statement_inside_loop>
                    | <assignment_inside_loop>
                    | <if_statement>

<yaz_statement_inside_loop> ::= "yaz" "(" <expression> ")"

<assignment_inside_loop> ::= <identifier> "=" <expression>

<assignment_no_semicolon> ::= <identifier> "=" <expression>

<condition_expression> ::= <expression> <comparison_operator> <expression>

<expression> ::= <term> { ("+" | "-") <term> }

<term> ::= <factor> { ("*" | "/" ) <factor> }

<factor> ::= <identifier>
          | <integer_literal>
          | <string_literal>
          | "(" <expression> ")"

<comparison_operator> ::= "==" | "!=" | "<" | ">" | "<=" | ">="

<identifier> ::= TOKEN_IDENTIFIER
<integer_literal> ::= TOKEN_INTEGER_LITERAL
<string_literal> ::= TOKEN_STRING_LITERAL
<EOF> ::= TOKEN_EOF

```

Note: The grammar correctly reflects operator precedence — multiplication and division bind more tightly than addition and subtraction, as implemented in the `evaluateExpression()`, `evaluateTerm()`, and `evaluateFactor()` methods of the `Parser` class.

6. Lexical Analyzer Design

Lexer.java — Class Explanation

This class performs lexical analysis — it reads the source code character by character and converts it into a list of tokens. It is located in the `PLProject` package.

Fields

- `String source`: The source code to be analyzed.
- `int currentPosition`: Tracks the current scanning position in the source string.
- `int currentLine, currentColumn`: Track line and column numbers for error reporting.

- `List<Token> tokens`: Stores the generated tokens.
- `Map<String, TokenType> KEYWORDS`: A static `HashMap` that maps keyword strings ("yaz", "her", "sürece", "eğer", "değilse") to `TOKEN_KEYWORD` type.

Functions

Lexer(String source)

Constructor. Initializes the source code string to be lexed.

char advance()

Moves to the next character and updates line/column counters. Returns the character read. Returns `'\0'` at end of input.

char peek()

Peeks at the current character without consuming it. Returns `'\0'` at end of input.

char peekNext(int offset)

Peeks ahead by a given offset. Useful for multi-character operator lookahead.

void skipWhitespace()

Skips whitespace characters including spaces, tabs, and newlines.

List<Token> lex()

The main lexical analyzer function. Loops over each character of the source, recognizing tokens such as keywords, operators, identifiers, numbers, and string literals. Handles Turkish characters. At the end, adds an EOF token. Returns the list of tokens.

Token handleIdentifierOrKeyword(int line, int column)

Reads a sequence of letters/digits and checks the `KEYWORDS` map. If the word is a keyword, creates a `TOKEN_KEYWORD`; otherwise, creates a `TOKEN_IDENTIFIER`.

Token handleNumber(int line, int column)

Reads a sequence of digits and creates a `TOKEN_INTEGER_LITERAL` token.

Token handleStringLiteral(int line, int column)

Reads characters between double quotes and creates a `TOKEN_STRING_LITERAL` token. Throws an error if the closing quote is missing.

Token handleOperator(int line, int column)

Handles single and compound operators. Checks if the current and next characters form a two-character operator (`==`, `!=`, `<=`, `>=`). Returns the appropriate `TOKEN_OPERATOR`.

boolean isCompoundOperatorStart(char c)

Returns true if the character can start a compound operator (`=`, `!`, `<`, `>`).

boolean isSingleCharOperator(char c)

Returns true if the character is a single-character operator (+, -, *, /).

boolean isSeparator(char c)

Returns true if the character is a separator ((,), {, }, ;).

7. Syntax Analyzer Design

Parser.java — Class Explanation

This class performs syntax analysis — it reads the tokens from the lexer and interprets them according to the LCS language grammar rules using a recursive-descent approach. It also performs basic interpretation (execution) of the parsed code.

Fields

- `List<Token> tokens`: The list of tokens to parse.
- `int currentTokenIndex`: The current position in the token list.
- `Map<String, Object> environment`: A `HashMap` that stores variable names and their values, simulating a variable environment for interpretation.

Functions

Parser(List<Token> tokens)

Constructor. Initializes the parser with a list of tokens.

Token currentToken()

Returns the current token in the list.

Token peekNextToken()

Looks ahead to the next token without advancing the index.

Token consumeToken()

Advances to the next token, unless it is EOF. Returns the consumed token.

Token match(TokenType expectedType)

Consumes the token only if its type matches the expected type; throws a syntax error otherwise.

Token match(TokenType expectedType, String expectedValue)

Same as above, but checks both token type and specific value.

void parse()

The main entry point for parsing. Loops through all tokens and parses statements one by one until EOF.

void parseStatement()

Parses a single statement. Dispatches to the appropriate handler based on the current token: yaz → parseYazStatement(), her → parseForLoopStatement(), sürece → parseWhileLoopStatement(), eğer → parseIfStatement(), identifier → parseAssignmentStatement().

void requireStatementTerminator(String statementType)

Checks that the current token is TOKEN_UNDERSCORE (_) and consumes it. Throws an error if the statement terminator is missing.

void parseYazStatement()

Parses a yaz (print) statement: matches "yaz", "(", evaluates the expression, matches ")", and requires the "_" terminator. Prints the evaluated value.

void parseYazStatementInsideLoop()

Same as parseYazStatement() but without requiring the "_" terminator (used inside loop and block bodies).

void parseAssignmentStatement()

Parses an assignment: matches identifier, "=", evaluates the expression, stores the value in the environment, and requires the "_" terminator.

void parseAssignmentStatementInsideLoop()

Same as parseAssignmentStatement() but without requiring the "_" terminator.

void parseAssignmentStatementNoSemicolon()

Parses an assignment for use inside for-loop headers (no terminator needed).

void parseForLoopStatement()

Parses a "her" (for) loop: matches "her", "(", initialization assignment, ";", condition, ";", increment assignment, ")", "{", body statements, "}". Evaluates the loop by re-parsing the condition and body on each iteration.

void parseWhileLoopStatement()

Parses a "sürece" (while) loop: matches "sürece", "(", condition, ")", "{", body statements, "}". Re-evaluates the condition before each iteration.

void parseIfStatement()

Parses an "eğer" (if) statement with optional "değilse" (else) block. Evaluates the condition and executes only the appropriate branch, skipping the other.

void skipBlock()

Skips over a block by counting brace depth. Used to skip false branches of if/else.

void parseBlockInnerStatement()

Parses a statement inside a block (if/else body). Dispatches to `yaz`, assignment, or nested if statements.

boolean evaluateCondition()

Evaluates a condition expression: evaluates left side, reads comparison operator, evaluates right side, and returns the boolean result.

void parseConditionStructure()

Parses a condition without evaluating it (used for initial for-loop setup to advance past condition tokens).

Object evaluateExpression()

Evaluates expressions with `+` and `-` operators. Handles both integer arithmetic and string concatenation. Calls `evaluateTerm()` for higher-precedence operations.

Object evaluateTerm()

Evaluates expressions with `*` and `/` operators. Handles multiplication and integer division. Calls `evaluateFactor()` for atomic values.

Object evaluateFactor()

Evaluates atomic values: identifiers (looked up in environment), integer literals, string literals, and parenthesized expressions.

Token.java — Class Explanation

Represents a single token with type, value, and source location information.

Fields

- `TokenType type`: The category of token (keyword, identifier, operator, etc.).
- `String value`: The string value of the token.
- `int line`: The line number in the source code (0-indexed internally, displayed as 1-indexed).
- `int column`: The column number (0-indexed internally, displayed as 1-indexed).

Constructor: `Token(TokenType type, String value, int line, int column)` — Initializes a new token with type, value, and position info.

toString(): Provides a formatted string representation of the token showing type, value, line, and column. Used for debugging and display.

TokenType.java — Class Explanation

The TokenType enum defines the different types of tokens that can be recognized during lexical analysis. This classification helps the Lexer and Parser understand the role of each token in the source code.

Token Type	Description
TOKEN_KEYWORD	Reserved words with predefined meaning: yaz, her, sürece, eğer, değilse
TOKEN_IDENTIFIER	Variable names defined by the programmer (e.g., sayi, mesaj)
TOKEN_INTEGER_LITERAL	Numeric constants (e.g., 42, 100, 0)
TOKEN_STRING_LITERAL	Text enclosed in quotes (e.g., "Merhaba", "Toplam: ")
TOKEN_OPERATOR	Symbols for operations: +, -, *, /, =, ==, !=, <, >, <=, >=
TOKEN_SEPARATOR	Punctuation: (,), {, }, ;
TOKEN_EOF	End of File — marks the end of the input stream
TOKEN_UNDERSCORE	The underscore _ character — used as the statement terminator

Main.java — Class Explanation

This is the main entry point class that coordinates the reading of source code, tokenization (lexical analysis), parsing (syntax analysis), and execution of LCS programs.

Execution Flow

Step	Action
1. Determine File Path	If a command-line argument is provided, it uses that as the source file path. Otherwise, it defaults to "ornek1.txt".
2. Read Source Code	Reads the content of the .txt file as a UTF-8 string using Files.readString(). If reading fails, an error message is printed

	and the program exits.
3. Lexical Analysis	Creates a Lexer instance with the source code and calls lex() to generate a list of tokens. Prints each token. If an error occurs (e.g., unrecognized character), it prints a "Leksik Analiz Hatası" and exits.
4. Syntax Analysis & Execution	Creates a Parser instance with the token list and calls parse(). The parser checks syntax and interprets the code (executes loops, assignments, print statements). If an error occurs, it prints the error message.

Console Output Structure

Throughout the program, formatted output is printed:

- --- Kaynak Kod --- : Prints the raw source code
- --- Belirteçler (Tokenlar) --- : Prints all recognized tokens
- --- Ayırıştırma Çıktısı --- : Shows parsing and execution output

8. State Diagrams

The following state diagrams illustrate the important lexical and syntactic structures of the LCS language. Each diagram shows the transitions between states for recognizing different token types and language constructs.

Identifier Recognition

Lexical Analysis: Identifier / Keyword Recognition

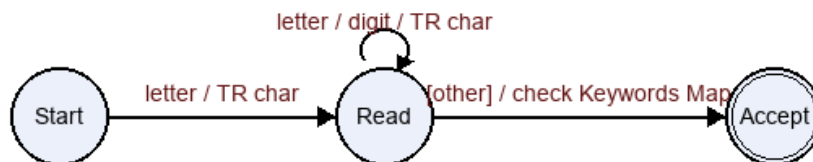


Figure: State Diagram for Identifier Recognition

Integer Literal Recognition

Lexical Analysis: Integer Literal Recognition

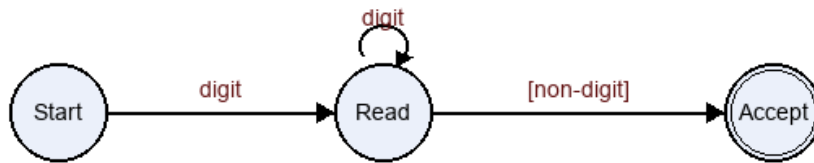


Figure: State Diagram for Integer Literal Recognition

String Literal Recognition

Lexical Analysis: String Literal Recognition

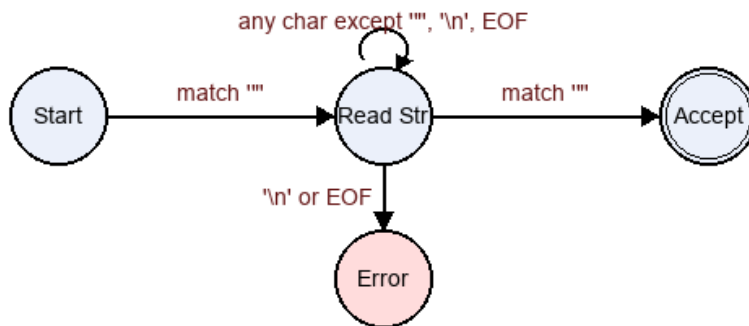


Figure: State Diagram for String Literal Recognition

Operator Recognition

Lexical Analysis: Operator Recognition

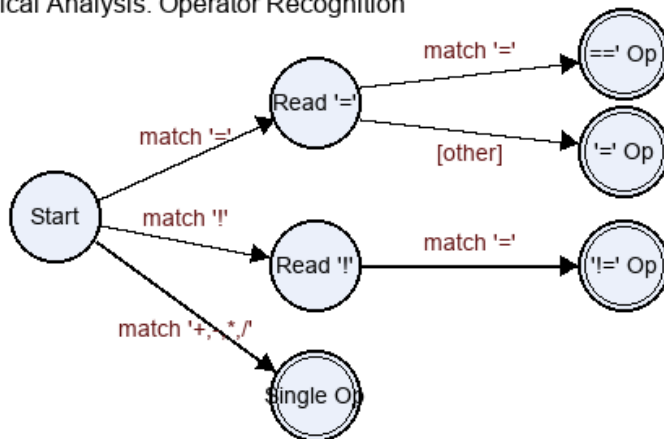


Figure: State Diagram for Operator Recognition

Conditional Structure (eğer/değilse)

Syntactic Analysis: Conditional Statement (eğer / değilse)

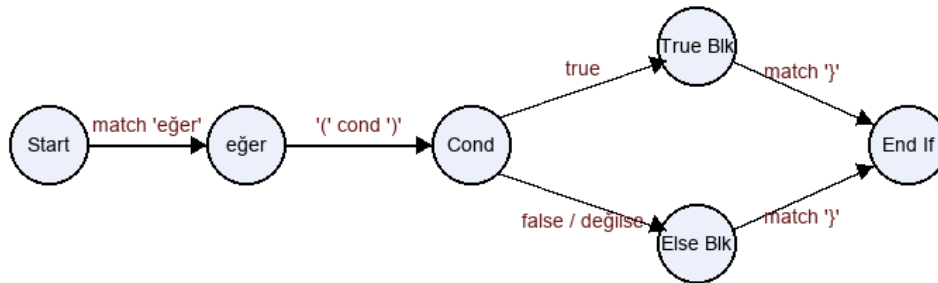


Figure: State Diagram for Conditional Structure (eğer/değilse)

Loop Structure (süreçe)

Syntactic Analysis: Loop Statement (süreçe)

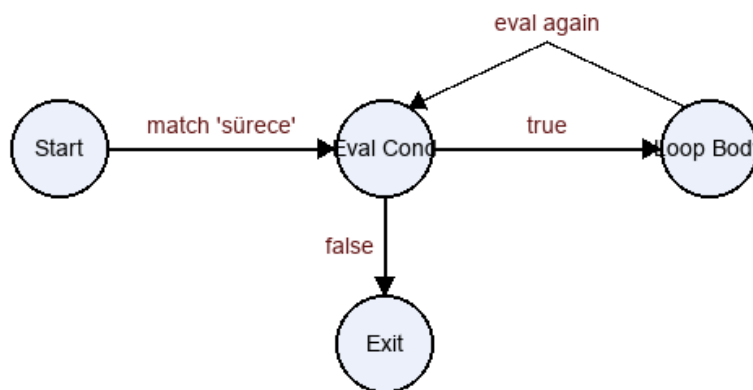


Figure: State Diagram for Loop Structure (süreçe)

9. Symbol Table / Lookup Table

The LCS language implementation uses two lookup mechanisms:

1. Keyword Lookup Table (in Lexer.java)

A static `HashMap<String, TokenType>` called `KEYWORDS` is used to distinguish keywords from identifiers during lexical analysis. When the lexer reads a word, it checks this map. If the word is found, it is classified as `TOKEN_KEYWORD`; otherwise, as `TOKEN_IDENTIFIER`.

Key (Lexeme)	Value (TokenType)
"yaz"	TOKEN_KEYWORD
"her"	TOKEN_KEYWORD
"sürece"	TOKEN_KEYWORD
"eğer"	TOKEN_KEYWORD
"değilse"	TOKEN_KEYWORD

2. Variable Environment (in Parser.java)

A `HashMap<String, Object>` called environment is used during parsing/interpretation to store variable names and their values. When a variable is assigned, it is stored in this map. When a variable is referenced in an expression, its value is looked up from this map.

The environment supports:

- Storing integer values (as Integer objects)
- Storing string values (as String objects)
- Dynamic typing — the same variable can hold different types
- Updating existing variables with new values

10. Sample Programs

Below are five source files written in the LCS programming language. Four are valid programs demonstrating different features, and one is an invalid program to test error detection.

Example 1 — Variable Declarations and String Concatenation

Filename: `ornek1.txt`

Source Code:

```
ad = "Kürşat"_
soyad = "Kılıçarslan"_
yas = 23_
puan = 85_

yaz (ad)_
yaz (soyad)_
yaz (yas)_
yaz (puan)_
```

```
tamAd = ad + " " + soyad_  
yaz (tamAd)_
```

Explanation:

- Defines string variables (ad, soyad) and integer variables (yas, puan)
- Prints each variable using the yaz() function
- Demonstrates string concatenation with the + operator (tamAd = ad + " " + soyad)
- Shows that the _ character is used as the statement terminator

Expected Output:

```
>>> Çıktı: Kürşat  
>>> Çıktı: Kılıçarslan  
>>> Çıktı: 23  
>>> Çıktı: 85  
>>> Çıktı: Kürşat Kılıçarslan
```

Example 2 — Arithmetic Expressions and Operator Precedence

Filename: ornek2.txt

Source Code:

```
toplam = 25 + 17_  
fark = 100 - 37_  
carpim = 8 * 6_  
bolum = 100 / 4_  
  
yaz (toplam)_  
yaz (fark)_  
yaz (carpim)_  
yaz (bolum)_  
  
sonuc = 100 - 20 * 3_  
yaz (sonuc)_  
  
sonuc2 = (2 + 3) * (4 + 1)_  
yaz (sonuc2)_
```

Explanation:

- Performs basic arithmetic: addition (25+17=42), subtraction (100-37=63), multiplication (8*6=48), division (100/4=25)
- Demonstrates operator precedence: 100 - 20 * 3 = 40 (multiplication before subtraction)
- Shows parenthesized expressions: (2+3) * (4+1) = 5 * 5 = 25
- Verifies that the parser correctly implements evaluateExpression → evaluateTerm → evaluateFactor

Expected Output:

```
>>> Çıktı: 42
>>> Çıktı: 63
>>> Çıktı: 48
>>> Çıktı: 25
>>> Çıktı: 40
>>> Çıktı: 25
```

Example 3 — Conditional Statements (eğer / değilse)

Filename: ornek3.txt

Source Code:

```
x = 10_
eğer (x > 5) {
    yaz ("x 5 ten büyük")
}

puan = 75_
eğer (puan >= 50) {
    yaz ("Gecti")
} değilse {
    yaz ("Kaldi")
}

a = 42_
b = 42_
eğer (a == b) {
    yaz ("a ve b esit")
} değilse {
    yaz ("a ve b esit degil")
}
```

Explanation:

- Demonstrates a simple if-statement: checks if $x > 5$ (true, since $x=10$)
- Shows if/else with *eğer/değilse*: checks if $puan \geq 50$ (true, since $puan=75$) → prints "Gecti"
- Tests equality comparison: $a == b$ (true, both are 42) → prints "a ve b esit"
- Note: Statements inside blocks do not require the "_" terminator

Expected Output:

```
>>> Çıktı: x 5 ten büyük
>>> Çıktı: Gecti
>>> Çıktı: a ve b esit
```

Example 4 — Loop Structures (her / sürece)

Filename: ornek4.txt

Source Code:

```
toplam = 0_
her (i = 1; i <= 5; i = i + 1) {
    toplam = toplam + i
}
yaz ("Toplam: " + toplam)_

sayac = 3_
sürece (sayac > 0) {
    yaz (sayac)
    sayac = sayac - 1
}
yaz ("Bitti")_
```

Explanation:

- Demonstrates the "her" (for) loop: iterates i from 1 to 5, summing values (1+2+3+4+5=15)
- Shows string concatenation with an integer result: "Toplam: " + 15 → "Toplam: 15"
- Demonstrates the "sürece" (while) loop: counts down from 3 to 1
- Loop variables are updated inside the loop body without the "_" terminator

Expected Output:

```
>>> Çıktı: Toplam: 15
>>> Çıktı: 3
>>> Çıktı: 2
>>> Çıktı: 1
>>> Çıktı: Bitti
```

Example 5 — Invalid Program (Syntax Error)

Filename: ornek5.txt

Source Code:

```
yaz ("Bu satir dogru yazilmis")_

x = 10_

yaz (x_

yaz ("Bu satira hic ulasilamaz")_
```

Explanation:

- The first two statements are valid and execute correctly
- Line 5 contains a syntax error: yaz (x_ — the closing parenthesis ")" is missing before the "_" terminator
- The parser detects this error and reports: "Beklenen token → ')' ama bulunan token → ' '_"

- The last line is never reached due to the error on line 5
- This demonstrates the error detection capabilities of the parser

Expected Output:

```
>>> Çıktı: Bu satır dogru yazilmis
>>> Atama: x = 10
>>> Çıktı: 10
Syntax Hatası: Beklenen token -> ' ) ' ama bulunan token -> ' _ ' 5. satır, 7. sütun.
```

11. Test Results

All five sample programs were compiled and tested using the LCS interpreter. The following sections show the actual outputs when running each program.

Compilation command: `javac -encoding UTF-8 PLProject/*.java`

Run command: `java PLProject.Main <filename.txt>`

ornek1.txt — Result: SUCCESS ✓

```
--- Kaynak Kod ---
ad = "Kürşat"_
soyad = "Kılıçarslan"_
yas = 23_
puan = 85_

yaz (ad)_
yaz (soyad)_
yaz (yas)_
yaz (puan)_

tamAd = ad + " " + soyad_
yaz (tamAd)_
-----

--- Ayırıştırma Çıktısı ---
>>> Atama: ad = Kürşat
>>> Atama: soyad = Kılıçarslan
>>> Atama: yas = 23
>>> Atama: puan = 85
>>> Çıktı: Kürşat
>>> Çıktı: Kılıçarslan
>>> Çıktı: 23
>>> Çıktı: 85
>>> Atama: tamAd = Kürşat Kılıçarslan
>>> Çıktı: Kürşat Kılıçarslan
```

Ayırıştırma ve çalıştırma başarıyla tamamlandı.

ornek2.txt — Result: SUCCESS ✓

```
--- Ayrıştırma Çıktısı ---
>>> Atama: toplam = 42
>>> Atama: fark = 63
>>> Atama: carpim = 48
>>> Atama: bolum = 25
>>> Çıktı: 42
>>> Çıktı: 63
>>> Çıktı: 48
>>> Çıktı: 25
>>> Atama: sonuc = 40
>>> Çıktı: 40
>>> Atama: sonuc2 = 25
>>> Çıktı: 25
```

Ayrıştırma ve çalıştırma başarıyla tamamlandı.

ornek3.txt — Result: SUCCESS ✓

```
--- Ayrıştırma Çıktısı ---
>>> Atama: x = 10
Koşul DOĞRU – eğer bloğu çalıştırılıyor.
>>> Çıktı: x 5 ten büyük
>>> Atama: puan = 75
Koşul DOĞRU – eğer bloğu çalıştırılıyor.
>>> Çıktı: Gecti
Koşul DOĞRU olduğu için değilse bloğu atlanıyor.
>>> Atama: a = 42
>>> Atama: b = 42
Koşul DOĞRU – eğer bloğu çalıştırılıyor.
>>> Çıktı: a ve b esit
Koşul DOĞRU olduğu için değilse bloğu atlanıyor.
```

Ayrıştırma ve çalıştırma başarıyla tamamlandı.

ornek4.txt — Result: SUCCESS ✓

```
--- Ayrıştırma Çıktısı ---
>>> Atama: toplam = 0
>>> Döngü Başlığı Ataması: i = 1
>>> Atama: toplam = 1
>>> Döngü Başlığı Ataması: i = 2
>>> Atama: toplam = 3
>>> Döngü Başlığı Ataması: i = 3
>>> Atama: toplam = 6
>>> Döngü Başlığı Ataması: i = 4
>>> Atama: toplam = 10
>>> Döngü Başlığı Ataması: i = 5
>>> Atama: toplam = 15
>>> Döngü Başlığı Ataması: i = 6
>>> Çıktı: Toplam: 15
>>> Atama: sayac = 3
```

```
>>> Çıktı: 3
>>> Atama: sayac = 2
>>> Çıktı: 2
>>> Atama: sayac = 1
>>> Çıktı: 1
>>> Atama: sayac = 0
>>> Çıktı: Bitti
```

Ayrıştırma ve çalıştırma başarıyla tamamlandı.

ornek5.txt — Result: ERROR DETECTED ✓

```
--- Ayrıştırma Çıktısı ---
>>> Çıktı: Bu satir dogru yazilmis
>>> Atama: x = 10
>>> Çıktı: 10
```

Syntax Hatası: Beklenen token -> ') ' ama bulunan token -> ' _ ' 5. satır, 7. sütun.

Test Summary

File	Feature Tested	Result	Notes
ornek1.txt	Variables, strings, concatenation	PASS ✓	All outputs correct
ornek2.txt	Arithmetic, operator precedence	PASS ✓	100-20*3=40 (correct precedence)
ornek3.txt	If/else conditionals	PASS ✓	All conditions evaluated correctly
ornek4.txt	For and while loops	PASS ✓	Loop iterations correct
ornek5.txt	Syntax error detection	PASS ✓	Missing) detected with line/column info

12. Conclusion

In this project, we designed and implemented a small programming language called LCS (LanguageCompileStructure) with Turkish-inspired keywords, and built a working lexical analyzer and recursive-descent parser for it in Java.

Base Textbook Code Used

We used the textbook examples from Robert W. Sebesta, "Concepts of Programming Languages", 10th Edition, Chapter 4 (Lexical and Syntax Analysis) as our foundation:

What We Kept:

- The character-by-character scanning approach for lexical analysis
- The recursive-descent parsing technique where each grammar rule maps to a function
- The overall architecture: Lexer → Token list → Parser
- The concept of token types for classifying source code elements

What We Modified:

- Converted the implementation from C to Java (object-oriented design)
- Replaced English keywords with Turkish keywords (yaz, her, sürece, eğer, değilse)
- Changed the statement terminator from semicolon to underscore (_)
- Added Turkish character support in the lexer (ç, ğ, ı, ö, ş, ü)
- Implemented proper operator precedence with three levels (expression → term → factor)
- Added line and column tracking for meaningful error messages

What We Added:

- String literal support with concatenation
- For-loop (her) and while-loop (sürece) implementations with re-parsing for iteration
- If/else (eğer/değilse) conditional statements with block skipping
- A variable environment (HashMap) for basic interpretation and execution
- Compound operator recognition (==, !=, <=, >=)
- Error recovery and Turkish error messages
- Multiple parsing functions for different contexts (top-level vs. inside loops/blocks)

The project successfully demonstrates the connection between programming language theory and practical implementation. All four valid sample programs run correctly, and the invalid program is properly detected with a meaningful error message including line and column information. The lexer correctly handles Turkish characters, and the parser implements a complete recursive-descent strategy that matches the BNF grammar definition.